

编译原理实验一实验报告

吴季鸿 241220142 241220142@smail.nju.edu.cn

一、程序实现的功能

本程序实现了 C++ 的词法分析与语法分析。

在词法分析部分，程序能够识别关键字、标识符、整数、浮点数、运算符和界符，并能检测错误类型 A，即未定义字符和不符合语法规则的单词。例如，输入中出现 ~ 时，程序会输出对应的词法错误信息。

当出现这类错误时，程序按要求输出：

```
Error type A at Line x: ...
```

在语法分析部分，程序能够识别变量定义、函数定义、复合语句、条件语句、循环语句、表达式、数组访问和函数调用等结构，并在输入合法时输出语法树；当输入不符合语法规则时，能够输出错误类型 B。例如，对 `a[5,3]` 可报缺少 `]`，对 `if (...) i = 1 else ...` 可报缺少 `;`。

此外，本程序按照任务号完成了选做要求 2.1，能够识别八进制数和十六进制数。对于合法输入如 `0123`、`0x3F`，程序可正确识别；对于非法输入如 `09`、`0x3G`，程序会输出相应的词法错误信息。

二、程序实现说明

词法分析器使用 Flex 实现，主要在 `lexical.l` 中通过正则表达式描述各类词法单元，并增加了对八进制数、十六进制数及其非法形式的识别规则。为了避免一个非法数字同时引发额外语法错误，程序在报告词法错误后仍返回占位的整数记号，从而保证分析过程继续进行。

语法分析器使用 Bison 实现，主要在 `syntax.y` 中定义文法规则，并在归约时构造语法树结点。对于缺少分号、缺少右中括号等常见错误，程序加入了简单的错误恢复规则，以便继续分析后续输入并输出更符合要求的错误信息。

由于实验提供的 `Makefile` 不能修改，且最终链接时不会链接 `lex.yy.o`，本程序将 `lex.yy.c` 直接包含进 `syntax.y` 生成的文件中，以保证编译能够顺利通过。

三、程序编译方法

本程序使用实验给定的 `Makefile` 编译。在实验目录下执行：

```
make
```

编译成功后会生成可执行文件 `parser`。运行方式为：

```
./parser <源文件路径>
```

例如：

```
./parser ../Test/test1.cmm  
./parser ../Test/test2.cmm  
./parser ../Test/test6.cmm
```

也可以使用：

```
make test
```

对测试文件进行测试。